Empirical Evaluation of Retrieval Strategies in Retrieval-Augmented Generation

Damilola Olaiya DAMILOLAOLAIYA@CMAIL.CARLETON.CA
Data Science, Analytics and Artificial Intelligence
Carleton University
Ottawa, ON, Canada

Abstract

Retrieval-Augmented Generation (RAG) augments large language models with external knowledge by retrieving relevant passages before generation. The retrieval strategy—dense, sparse, or hybrid—strongly affects retrieval quality, which I use as a proxy for downstream grounding quality. I empirically compare these strategies on five BEIR datasets (nfcorpus, fiqa, scifact, arguana, dbpedia-entity), with chunk granularity variation (original, 128, 256, 512). I run two full grids that differ by dense encoder backbone (all-MiniLM-L6-v2 vs all-mpnet-base-v2) and compare retrieval metrics (MRR, Recall@k, NDCG@k). Hybrid retrieval via Reciprocal Rank Fusion is the strongest overall performer. Across these runs, dense-encoder swapping has negligible effect on aggregate retrieval quality, while chunk size and retrieval-family choice remain the dominant factors. The code is available here on GitHub.

Keywords: retrieval-augmented generation, dense retrieval, sparse retrieval, BM25, hybrid retrieval, chunk size, BEIR benchmark

1 Introduction

1.1 Problem Statement

Retrieval-Augmented Generation (RAG) augments large language models (LLMs) with external knowledge by retrieving relevant passages before generation (Lewis et al., 2020). Given a user query, a RAG system first retrieves passages from a corpus, then conditions the LLM on those passages to produce a grounded answer. This paradigm reduces hallucination and enables knowledge-grounded question answering without retraining the model.

The retrieval stage is often the critical bottleneck: if retrieval fails to surface relevant passages, the LLM receives irrelevant context and produces degraded or incorrect answers. Despite widespread adoption, there is limited systematic comparison of how different retrieval approaches perform under varying conditions—domain, corpus size, query type, and document granularity. Practitioners lack clear guidance on which strategy to select for a given use case.

1.2 Importance of the Problem

RAG is deployed in enterprise knowledge bases, customer support systems, and research assistants. Suboptimal retrieval leads to poor user experience, factual errors, and wasted compute. Understanding the relative strengths of dense retrieval (semantic matching via learned embeddings), sparse retrieval (lexical matching via BM25), and hybrid approaches

©2026 Damilola Olaiya.

License: CC-BY 4.0, see https://creativecommons.org/licenses/by/4.0/.

is essential for building effective RAG systems. Furthermore, the optimal retrieval method often shifts depending on the length and granularity of retrieved chunks—short sentences versus full paragraphs—a dimension I explicitly analyze.

2 Techniques to Tackle the Problem

2.1 Review of Previous Work

Dense retrieval encodes queries and passages into a shared vector space and retrieves via approximate nearest-neighbor search. Karpukhin et al. (2020) introduced Dense Passage Retrieval (DPR), demonstrating that learned dual-encoders outperform sparse methods on open-domain QA. Lewis et al. (2020) combined DPR with a generative model in the RAG architecture. Izacard and Grave (2021) extended this with FiD (Fusion-in-Decoder). Guu et al. (2020) pre-trained retrieval-augmented language models. Dense retrieval excels at semantic matching but can underperform on lexical or domain-specific terminology.

Sparse retrieval relies on term frequency and inverse document frequency. BM25 (Robertson and Zaragoza, 2009) remains a strong baseline, especially for keyword-heavy queries and when training data for dense models is limited. It requires no neural components and runs efficiently on CPU.

Hybrid retrieval combines dense and sparse methods. Reciprocal Rank Fusion (RRF) merges ranked lists without score calibration. Gao et al. (2023) survey RAG for LLMs and note that hybrid approaches often improve robustness. Khattab and Zaharia (2020) propose late-interaction retrieval (ColBERT), which I do not implement in this project due to time and compute/resource constraints but discuss as potential future work.

Benchmarks. Thakur et al. (2021) introduce BEIR, a heterogeneous benchmark for zero-shot retrieval evaluation across 18 datasets. I run the full grid on five BEIR datasets: nfcorpus, fiqa, scifact, arguana, and dbpedia-entity.

2.2 Techniques Chosen and Rationale

I implement and compare three retrieval strategies:

(1) Dense retrieval using sentence-transformers (Reimers and Gurevych, 2019). I evaluate two encoder backbones in separate full-grid runs: all-MiniLM-L6-v2 and all-mpnet-base-v2. Documents and queries are encoded into dense vectors; retrieval uses cosine similarity.

(2) Sparse retrieval using BM25 with standard parameters ($k_1 = 1.5$, $b = 0.75$). Tokenization is word-based. No learned components; purely lexical.

(3) Hybrid retrieval via Reciprocal Rank Fusion of dense and sparse results. For each document d appearing in ranked lists L_1, L_2, the RRF score is

$$\text{score}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

with $k = 60$. This combines semantic and lexical signals without score calibration.

Chunk size variation. To evaluate how optimal retrieval depends on granularity, I vary chunk sizes: original documents, 128-token chunks, 256-token chunks, and 512-token chunks.

3 Empirical Evaluation

3.1 Experimental Setup

Datasets. I use BEIR benchmarks via ir_datasets: *nfcorpus*, *fiqa*, *scifact*, *arguana*, and *dbpedia-entity*. These datasets span different retrieval settings, including finance-oriented QA (fiqa), scientific claim verification (scifact), argument retrieval (arguana), and entity-centric retrieval (dbpedia-entity). For each dataset, I cap evaluation at 200 queries to keep the full grid tractable.

Metrics. I report Recall@k ($k \in \{1, 5, 10, 100\}$), MRR, and NDCG@k. Let R_q denote relevant documents for query q, $T_k(q)$ the top-k retrieved set, and rank_q the rank of the first relevant item. Then:

$$\text{Recall@}k = \frac{|R_q \cap T_k(q)|}{|R_q|}, \quad \text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q},$$

and NDCG@k is DCG@k/IDCG@k.

Implementation. I used Python with sentence-transformers, rank-bm25, and ir-datasets. Retrieval depth is fixed at top-$K = 100$; metrics are computed at $k = \{1, 5, 10, 100\}$. I compare two completed full-grid runs: MiniLM (run ID 20260328_191443) and MPNet (run ID 20260406_235803). For tractability, the query cap uses the first 200 valid queries returned by the dataset loader for each dataset. This report evaluates retrieval quality only; end-to-end LLM answer-generation quality is outside scope.

3.2 Results: Method Comparison

Table 1 and Table 2 summarize results on nfcorpus and scifact with original (unchunked) documents from run 20260328_191443.

Table 1: Retrieval performance on nfcorpus (original documents).

Method	MRR	R@1	R@5	R@10	NDCG@10	Time (s)
Dense	0.544	0.051	0.124	0.160	0.345	22.9
Sparse	0.559	**0.069**	0.134	0.167	0.349	**0.4**
Hybrid	**0.591**	0.066	**0.144**	**0.171**	**0.373**	24.2

Table 2: Retrieval performance on scifact (original documents).

Method	MRR	R@1	R@5	R@10	NDCG@10	Time (s)
Dense	0.606	0.476	0.731	0.772	0.637	32.5
Sparse	0.652	0.532	0.751	0.798	0.677	**2.1**
Hybrid	**0.676**	**0.544**	**0.777**	**0.828**	**0.703**	35.2

Provenance note: Table 1, Table 2, and Table 3 use run 20260328_191443 values; most diagnostic figures in this report are from run 20260406_235803. This is intentional because the cross-run quality deltas are minimal (Table 4), while latency is not compared across hardware environments.

Findings. Hybrid retrieval achieves the strongest retrieval quality on both showcased datasets. On nfcorpus, hybrid MRR is 0.591 (vs. 0.559 sparse, 0.544 dense). On scifact, hybrid MRR is 0.676 (vs. 0.652 sparse, 0.606 dense). However, the full-dataset view (Appendix A) also shows exceptions (e.g., arguana and fiqa) where dense ranks highest, so hybrid should be interpreted as the strongest overall default rather than a universal winner on every dataset slice. Sparse remains the fastest on this CPU-style run.

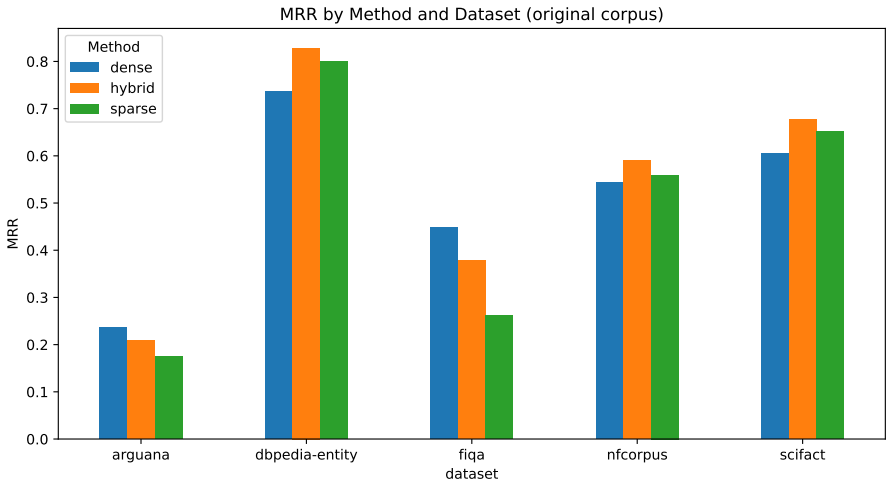

Figure 1: MRR by retrieval method and dataset (original documents, run 20260406_235803).

Figure 1 visually confirms the method-comparison trend used in my main tables: hybrid is the most consistent top performer across datasets, while dense and sparse trade places by domain.

3.3 Results: Chunk Size and Granularity

Table 3 shows how chunk size affects performance on nfcorpus (run 20260328_191443).

Findings. Chunk effects are method-dependent. For dense retrieval, original and 512-token chunks are close in MRR, while 128-token chunks improve NDCG@10. For sparse retrieval, 512-token chunks yield the best MRR. Hybrid remains strongest overall; 512-token chunks give the top hybrid MRR (0.601), while original is best on Recall@10.

Figure 2 illustrates the granularity sensitivity discussed in Table 3: method rankings and absolute scores vary with chunking choice.

3.4 Results: MiniLM vs MPNet Comparison

Table 4 compares aggregate quality metrics between the two completed full-grid runs, averaged across all datasets and chunk settings for each retrieval method.

Findings. Aggregate differences between MiniLM and MPNet are minimal in this experimental setup (typically around 10^{-4}). The dominant gains come from retrieval-family choice (hybrid vs single-family) and chunking configuration rather than encoder swap alone.

Table 3: Effect of chunk size on nfcorpus.

Method	Chunk	MRR	R@5	R@10	NDCG@10
Dense	original	**0.544**	**0.124**	0.160	0.345
	128 tok	0.539	0.083	0.116	**0.380**
	256 tok	0.533	0.098	0.138	0.349
	512 tok	0.540	0.119	**0.166**	0.343
Sparse	original	0.559	0.134	0.167	0.349
	128 tok	0.545	0.097	0.115	**0.366**
	256 tok	0.553	0.122	0.145	0.356
	512 tok	**0.569**	**0.138**	**0.167**	0.354
Hybrid	original	0.591	**0.144**	**0.171**	0.373
	128 tok	0.576	0.103	0.130	**0.405**
	256 tok	0.579	0.125	0.149	0.377
	512 tok	**0.601**	0.143	0.166	0.372

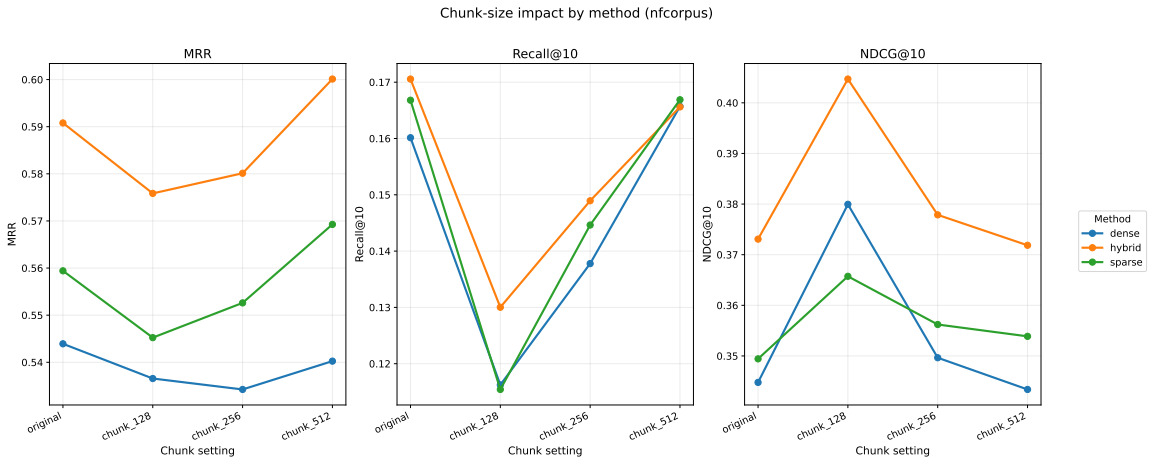

Figure 2: Chunk-size impact on nfcorpus for MRR, Recall@10, and NDCG@10 (run 20260406_235803).

Table 4: Embedding model comparison across all datasets/chunks (MiniLM = run A = 20260328_191443, MPNet = run B = 20260406_235803).

Method	MiniLM MRR	MPNet MRR	MiniLM R@10	MPNet R@10	MiniLM NDCG@10	MPNet NDCG@10
Dense	**0.51067**	0.51059	0.39367	0.39367	**0.37370**	0.37369
Sparse	**0.48840**	0.48836	**0.33963**	0.33963	**0.32335**	0.32334
Hybrid	0.53337	**0.53347**	**0.38356**	0.38355	0.37583	**0.37585**

Because the two runs were executed in different hardware environments, I use this section for quality-metric comparison only and avoid cross-run latency claims. Future work could explicitly log the active dense model in per-run metadata to strengthen provenance and reproducibility.

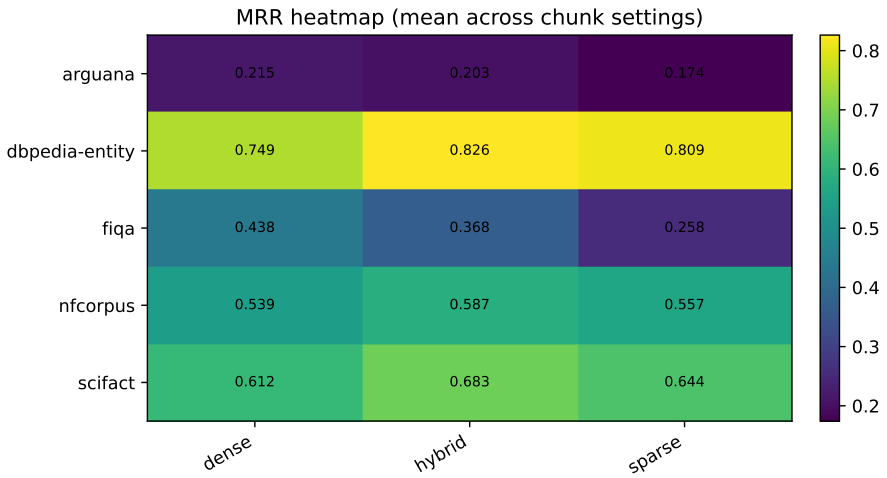

Figure 3: MRR heatmap (dataset × method) aggregated across chunk settings (run 20260406_235803).

Figure 3 provides a compact cross-dataset summary and supports the conclusion that retrieval-family choice (especially hybrid vs single-family) has larger practical impact than swapping between the two dense backbones in this setup.

Global summary across parameters. I ran a comprehensive sweep across all combinations of 5 datasets, 3 methods, 4 chunk settings, and fixed retrieval metrics. Due to the 8-page limit, the main body presents representative slices (Table 1, Table 2, Table 3) plus the aggregate embedding-model comparison (Table 4). Full dataset-level and chunk-level comparisons are provided in Appendix A and Appendix B, with additional diagnostics in Appendix C.

3.5 Complexity and Ease of Use

Computational complexity. Sparse retrieval uses lexical indexing and ranking; dense retrieval adds neural embedding computation and vector similarity search. Hybrid adds rank-fusion overhead on top of both retrieval passes.

Ease of use. Sparse retrieval has minimal dependencies and is easiest to deploy. Dense retrieval requires model loading and embedding infrastructure. Hybrid adds little implementation complexity beyond running both and fusing ranks.

4 Conclusion

4.1 Best Technique

Hybrid retrieval via Reciprocal Rank Fusion consistently outperforms dense-only and sparse-only baselines on key datasets in this study. On original-document runs, hybrid reaches MRR 0.591 on nfcorpus (vs. 0.559 sparse, 0.544 dense) and 0.676 on scifact (vs. 0.652 sparse, 0.606 dense). For practitioners, hybrid is the strongest default when both dense and sparse components are available.

4.2 Is the Problem Solved?

No. Retrieval remains a bottleneck. Even under the best settings here, many relevant items are still missed and performance varies by dataset and chunk configuration. Domain shift, long documents, and multi-hop reasoning remain challenging. Chunk size and granularity should be tuned per application.

4.3 Future Research

For future research work, I could look into: (1) end-to-end evaluation with an LLM to measure answer quality, not only retrieval proxies; (2) larger benchmarks (e.g., MS MARCO, NQ) and broader domain diversity; (3) inclusion of late-interaction and learned sparse methods (e.g., ColBERT, SPLADE) under matched hardware budgets; (4) learned chunking strategies that adapt granularity to query type; (5) ablations on max_queries, retrieval top-K, and RRF constant to quantify stability; and (6) structured failure-mode analysis for hybrid misses.

Appendix A. Dataset-Level MiniLM vs MPNet Comparison

This appendix reports method-level means per dataset using the same comparison pipeline outputs used in the main text. It supports the claim that MiniLM vs MPNet differences remain very small across the full parameter sweep.

Table 5: Per-dataset, per-method MiniLM vs MPNet comparison (MRR and Recall@10, rounded to 3 decimals).

Dataset	Method	MRR_A	MRR_B	ΔMRR	$R@10_A$	$R@10_B$	ΔR@10
arguana	dense	0.215	0.215	0.000	0.655	0.655	0.000
arguana	sparse	0.174	0.174	0.000	0.539	0.539	0.000
arguana	hybrid	0.203	0.203	0.000	0.603	0.603	0.000
dbpedia-entity	dense	0.749	0.749	0.000	0.046	0.046	0.000
dbpedia-entity	sparse	0.809	0.809	0.000	0.060	0.060	0.000
dbpedia-entity	hybrid	0.826	0.826	0.000	0.060	0.060	0.000
fiqa	dense	0.438	0.438	0.000	0.377	0.377	0.000
fiqa	sparse	0.258	0.258	0.000	0.205	0.205	0.000
fiqa	hybrid	0.368	0.368	0.000	0.321	0.321	0.000
nfcorpus	dense	0.539	0.539	0.000	0.145	0.145	0.000
nfcorpus	sparse	0.557	0.557	0.000	0.148	0.148	0.000
nfcorpus	hybrid	0.586	0.587	0.000	0.154	0.154	0.000
scifact	dense	0.612	0.612	0.000	0.745	0.745	0.000
scifact	sparse	0.644	0.644	0.000	0.746	0.746	0.000
scifact	hybrid	0.683	0.683	0.000	0.781	0.781	0.000

Appendix B. Chunk-Level Comparison (Representative Dataset)

This table shows the full chunk-level MiniLM vs MPNet comparison for nfcorpus (all methods and all chunk settings), illustrating how non-displayed parameters are tracked and compared.

Table 6: Chunk-level nfcorpus MiniLM vs MPNet comparison (MRR and Recall@10, rounded to 3 decimals).

Method	Chunk	MRR_A	MRR_B	ΔMRR	$R@10_A$	$R@10_B$	ΔR@10
dense	original	0.544	0.544	0.000	0.160	0.160	0.000
dense	chunk_128	0.539	0.537	0.003	0.116	0.116	0.000
dense	chunk_256	0.533	0.534	-0.001	0.138	0.138	0.000
dense	chunk_512	0.540	0.540	0.000	0.166	0.166	0.000
sparse	original	0.559	0.559	0.000	0.167	0.167	0.000
sparse	chunk_128	0.545	0.545	0.000	0.115	0.115	0.000
sparse	chunk_256	0.553	0.553	0.000	0.145	0.145	0.000
sparse	chunk_512	0.569	0.569	0.000	0.167	0.167	0.000
hybrid	original	0.591	0.591	0.000	0.171	0.171	0.000
hybrid	chunk_128	0.576	0.576	0.000	0.130	0.130	0.000
hybrid	chunk_256	0.579	0.580	-0.001	0.149	0.149	0.000
hybrid	chunk_512	0.601	0.600	0.000	0.166	0.166	0.000

Appendix C. Additional Figures

These additional plots provide coverage for non-displayed parameters and support the same central conclusions reported in the main body.

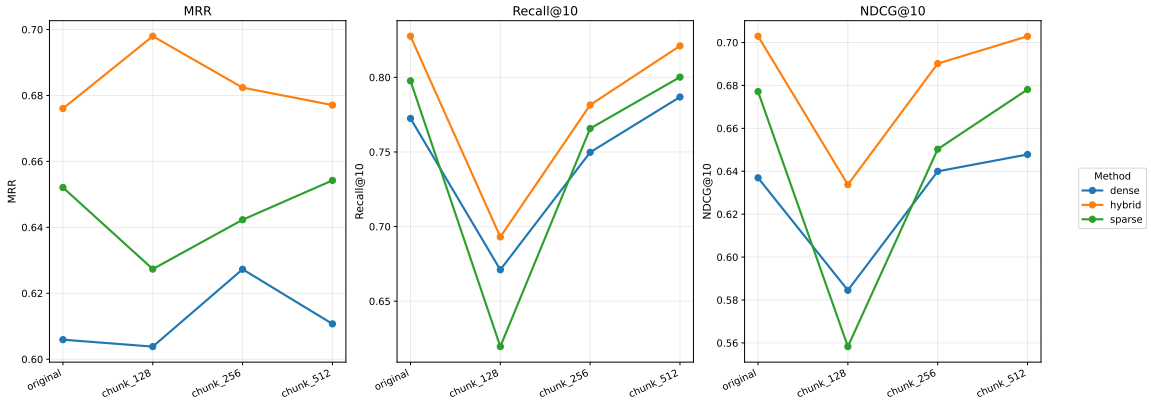

Figure 4: Chunk-size impact on scifact (MRR, Recall@10, NDCG@10) for run 20260406_235803.

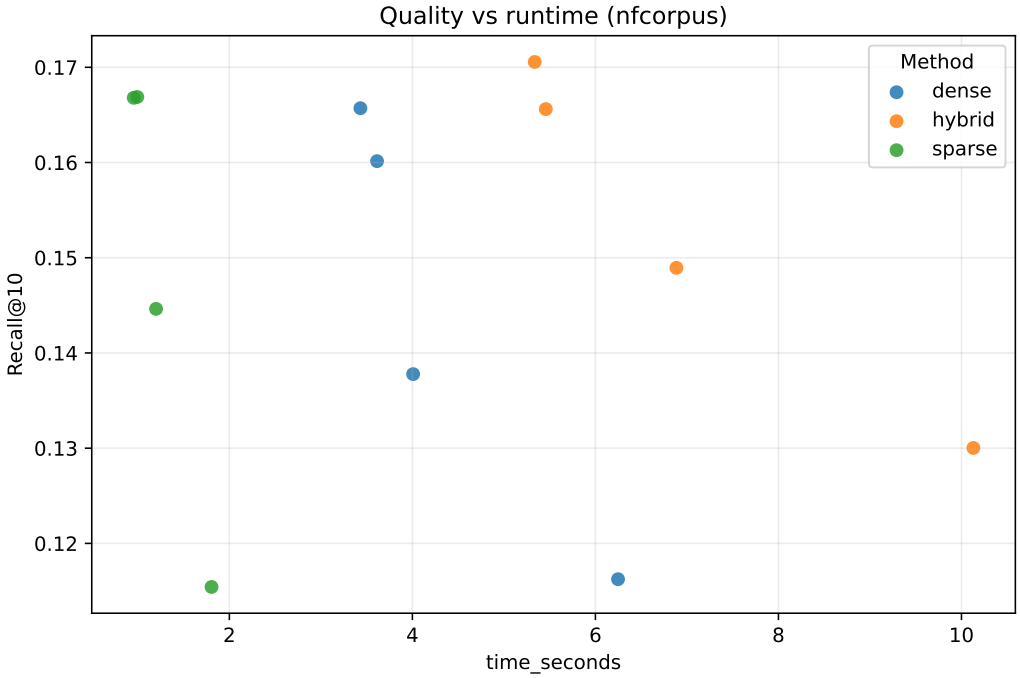

Figure 5: Runtime-quality tradeoff on nfcorpus for run 20260406_235803.

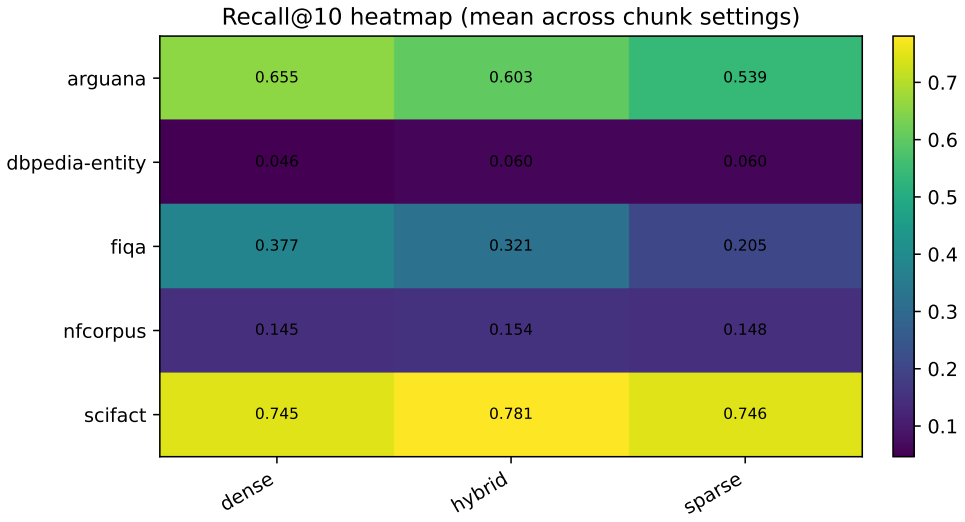

Figure 6: Recall@10 heatmap across datasets and methods for run 20260406_235803.

References

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Yi, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 3929–3938, 2020.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 874–880, 2021.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*, pages 1–13, 2021.